HƯỚNG DẪN SỬ DỤNG, KỊCH BẢN DEMO & LỜI THUYẾT TRÌNH

Hệ thống Quản lý Đào tạo EduFlaw (Web Pentest Console + Thanh nổi Web App) · Chuẩn OWASP Top 10:2025

Nhóm 7 — Khóa D24 (PTIT Hà Nội)
GVHD: **ThS. Đỗ Xuân Chợ**
Tài liệu thực hành & bảo vệ ngắn gọn

1. CÁCH KHỞI CHẠY NHANH (4 FILE LAUNCHER TẠI THƯ MỤC GỐC)

File khởi chạy	Khi nào dùng	Chức năng & Phím tắt điều khiển
`3_RESET_CSDL_DEMO.bat`	**Chạy trước buổi demo**	Xóa sạch & nạp lại MySQL `eduflaw_db` chuẩn (20 SV, 5 GV, đặt lại mật khẩu mẫu và mở hết khóa đăng nhập cũ).
`4_CHAY_KIEM_THU_PENTEST.bat`	**Demo chính (Khuyên dùng)**	Tự bật Flask tại cổng `5000` (nếu chưa chạy) và mở thẳng `http://127.0.0.1:5000/pentest?toolbar=1`. Phím trên cửa sổ console: `[O]` Mở lại `/pentest`, `[W]` Mở `/login`, `[C]` Chế độ Terminal CLI log cũ, `[Q]` Thoát. *(Truyền tham số `--cli` hoặc `1..7` nếu muốn chạy thuần Terminal).*
`1_CHAY_WEB_DEMO.bat`	Mở Web App thông thường	Khởi động Web App tại `http://127.0.0.1:5000/login` (có sẵn nút đăng nhập 1-click và link sang `/demo`, `/pentest?toolbar=1`).
`2_MO_LAB_THUYET_TRINH.bat`	Minh họa Slide lý thuyết	Mở phòng Lab mô phỏng trực quan 4 lỗ hổng (`demo/index.html` hoặc `/demo`). Phím tắt: `1–4` chọn bài, `V` đổi Vulnerable/Secure, `S` quét lại, `Enter` chạy mô phỏng.

2. TÀI KHOẢN DEMO CHUẨN & 2 KHU VỰC THAO TÁC TRỰC TIẾP TRÊN WEB APP

🔑 5 Tài Khoản Chuẩn (Dùng Nút 1-Click Không Cần Gõ)

Vai trò	Tài khoản	Mật khẩu
Master	`master@eduflaw.com`	`tung12345678`
Giảng viên 1	`gv.001@eduflaw.com`	`EduFlaw@2026`
Giảng viên 2	`gv.002@eduflaw.com`	`EduFlaw@2026`
Sinh viên 1	`b24dcat001@eduflaw.com`	`EduFlaw@2026`
Sinh viên 2	`b24dcat002@eduflaw.com`	`EduFlaw@2026`

* Mã SV tương ứng của SV1/SV2 là `B24DCAT001` / `B24DCAT002` (họ tên được tạo ngẫu nhiên mỗi lần reset CSDL).

Cấu Trúc Giao Diện `/pentest` & Thanh Nổi Web App

- **Thanh trên (Header `/pentest`):** Chip trạng thái MySQL, bộ đếm `Tiến độ: 0/6 bài`, công tắc **✓ Thanh nổi Web App: ĐANG BẬT**, nút **⚡ Chạy Tất Cả 6 Bài** và hàng nút **Đổi vai trò thật 1-Click** (👑 `Master`, 👩‍🏫 `GV1`, 👩‍🏫 `GV2`, `SV1`, `SV2`, 🚪 `Đăng xuất`) kèm **Mở trang vai trò hiện tại ↗**.
- **6 Tab kiểm thử + Nhật ký dưới:** Mỗi tab chạy request/SQL thật và có nút nhảy sang trang Web thật để kiểm chứng bằng mắt thường; mọi thao tác ghi log vào khung **Nhật ký Thực thi Trực tiếp**.
- **Thanh Pentest Nổi (góc dưới phải các trang Web thật):** Đứng ngay tại `/student/grades`, `/admin/grades`, `/master` vẫn đổi vai trò 1-click và bấm ngay 3 đòn A01 (**🎯 Thử IDOR ?student_id=2**, **💉 Thử vào /master**, **🧨 GV2 sửa lớp GV1**), **🔐 Mở khóa A07** hoặc **📊 Bảng Pentest Đầy Đủ ↗**.

Cơ chế an toàn của chế độ Pentest: Route `/pentest` và Thanh Pentest nổi chỉ hoạt động ở môi trường cục bộ (`localhost`) khi `APP_ENV` khác `production`; nếu cấu hình `production` hệ thống trả về `HTTP 404`, truy cập từ IP ngoài máy chủ trả `HTTP 403`.

3. LỘ TRÌNH TRÌNH DIỄN ĐỀ XUẤT (NGẮN GỌN ~8–10 PHÚT)

Phần	Nội dung chính	Thao tác chính trên màn hình	Thời lượng
Mở đầu	Khởi động & giới thiệu Thanh nổi trên Web thật	Mở `4_CHAY_KIEM_THU_PENTEST.bat` -> đổi sang `SV1` -> mở Web thật	~1 phút
Phần 1 (Bài 1)	**OWASP A01:** IDOR ngang, Leo thang dọc, Chéo GV	Thanh nổi Web thật + Tab `Bài 1 · OWASP A01`	~2 phút
Phần 2 (Bài 2)	**OWASP A03:** Chuỗi cung ứng SRI & `requirements.txt`	Tab `Bài 2 · OWASP A03` + mở F12 Console xem chặn SRI	~1.5 phút
Phần 3 (Bài 3)	**OWASP A07:** Brute-Force & Khóa kép 60s (429)	Tab `Bài 3 · OWASP A07` -> chạy 7 lần -> `Mở Khóa Nhanh`	~1.5 phút
Phần 4 & 5	**OWASP A10** (23 payload Fail-Closed) & **Quét 69 Endpoints**	Tab `Bài 4 · OWASP A10` & Tab `Bài 5 · Attack Surface`	~1.5 phút
Phần 6 (Chốt)	**DB Forensic:** CCCD AES-256-GCM & Sửa điểm thật ghi Audit Log	Tab `Bài 6` -> vào `/admin/grades` sửa điểm GV1 -> soi lại DB	~2 phút

4. KỊCH BẢN DEMO TỪNG PHẦN & LỜI THUYẾT TRÌNH (PHẦN MỞ ĐẦU, BÀI 1 & BÀI 2)

MỞ ĐẦU Khởi Động Hệ Thống & Kết Nối Trực Tiếp Lên Web App

⏱ ~1 phút

1. Nhấp đúp `4_CHAY_KIEM_THU_PENTEST.bat`. Khi trình duyệt tự mở `http://127.0.0.1:5000/pentest?toolbar=1`, chỉ chip xanh `● MySQL OK` và nút **✓ Thanh nối Web App: ĐANG BẬT**.
2. Trên hàng **Đổi vai trò thật 1-Click**, bấm **SV1** -> bấm **Mở trang vai trò hiện tại ↗** để mở thẳng giao diện Sinh viên thật kèm **Thanh Pentest Trực Tiếp** ở góc dưới phải màn hình.

> Lời thuyết trình mẫu:
> *"Kính thưa Thầy/Cô, để kiểm chứng bảo mật một cách trực quan nhất thay vì chỉ đọc log khô khan, nhóm em tích hợp Bảng điều khiển Pentest và Thanh công cụ nổi trực tiếp vào Web App EduFlaw khi chạy ở môi trường cục bộ. Thanh công cụ này chỉ bật ở local dev và tự động khóa 404 ở môi trường production. Em vừa chuyển sang phiên Sinh viên 1 thật trên trình duyệt để bắt đầu kiểm thử từng lỗ hổng theo chuẩn OWASP Top 10:2025."*

PHẦN 1 · BÀI 1 OWASP A01:2025 — Kiểm Soát Truy Cập, Chống IDOR & Phân Quyền 3 Cấp

⏱ ~2 phút

1. **1.1 Chống IDOR ngang (đọc trộm điểm SV khác):** Trên **Thanh nổi** ở trang Web thật, bấm **🎯 Thử IDOR ?student_id=2**. Trình duyệt đi tới URL `/student/grades?student_id=2` bằng phiên `SV1` nhưng bảng điểm hiển thị vẫn là của `SV1`. Tiếp tục bấm **📊 Bảng Pentest Đầy Đủ ↗**, tại mục `1.1` chọn preset `9999` hoặc `' OR 1=1--` rồi bấm **🚀 Gửi Request IDOR Thật** -> kết quả báo `AN TOÀN`, `Rò rỉ sinh viên khác: 0`.
2. **1.2 Chống leo thang dọc (`/master` , `/admin`):** Ở mục `1.2` bấm **Quét Route Quản Trị bằng SV1** -> toàn bộ 6 route quản trị đều trả `HTTP 302` và gắn nhãn `CHẶN THÀNH CÔNG`. Sau đó bấm **Mở thử /master bằng SV1 trên Web App ↗** (hoặc bấm **🚀 Thử vào /master** trên Thanh nổi) -> trình duyệt bị đẩy ngược về `/student` ngay lập tức.
3. **1.3 Chống can thiệp chéo giữa Giảng viên:** Tại mục `1.3` bấm **🧨 Gửi API Sửa Điểm Lén (GV2)** (hoặc bấm **🧨 GV2 sửa lớp GV1** trên Thanh nổi) -> gửi `POST /admin/grades/api/auto-save` dưới phiên `gv.002` với `grade_id` của lớp `gv.001` -> máy chủ từ chối với **HTTP 403**. *(Lưu ý giải thích nhanh: nếu GV2 mở `/admin/grades` bình thường thì hệ thống lọc sẵn nên không thấy lớp của GV1; API guard là lớp chốt chặn thứ hai nếu kẻ xấu tự sửa request).*

> Lời thuyết trình mẫu:
> *"Ở trọng tâm A01:2025, em kiểm chứng cả 3 chiều phân quyền trực tiếp trên phiên thật. Chiều ngang: dù truyền ?student_id=2 hay 9999 trên URL, hàm xem điểm chỉ lấy danh tính từ cookie session['user_id'] đã ký số HMAC-SHA256 nên không bao giờ xem nhầm điểm bạn khác. Chiều dọc: 6/6 route của Master và Giảng viên được bảo vệ bằng decorator @role_required, đẩy sinh viên về trang cá nhân với mã 302 thay vì xác nhận route tồn tại. Cuối cùng là chiều chéo vai trò: khi Giảng viên 2 cố gửi API sửa lén điểm lớp của Giảng viên 1, máy chủ kiểm tra ràng buộc lecturer_id == user_id và từ chối ngay với HTTP 403 Forbidden."*

PHẦN 2 · BÀI 2 OWASP A03:2025 — Chuỗi Cung Ứng Phần Mềm, W3C SRI & Dependency Pinning

⏱ ~1.5 phút

1. Chuyển sang tab **Bài 2 · OWASP A03**. Bấm **🔎 Kiểm Toán SRI & Requirements**. Chỉ mục `2.1` : công cụ bóc tách thẻ `<script>` tải `SweetAlert2` trên `/login`, tính SHA-256 thực tế từ CDN jsDelivr (khớp 100% với `integrity` đã ghim) và mô phỏng chèn thêm mã độc `navigator.sendBeacon(...)` làm mã băm sai lệch hoàn toàn. Bảng `2.3` bên dưới xác nhận **10/10 gói Python** trong `src/requirements.txt` đều ghim cứng phiên bản bằng toán tử `==`.
2. Mở DevTools trình duyệt (**F12 -> tab Console**). Tại mục `2.2` bấm **✅ Nạp Script với SRI ĐÚNG** -> trình duyệt nạp thành công (`onload()`). Ngay sau đó bấm **⛔ Nạp Script với SRI SAI 1 KÝ TỰ** (đổi ký tự cuối `I=` thành `X=`) -> trong Console lập tức hiện lỗi đỏ chặn thực thi, giao diện báo `onerror()` bảo vệ thành công. *(Hoặc bấm `Mở /Login để test DevTools F12 ↗` nếu muốn sửa trực tiếp trong tab Elements và nhấn `Ctrl + F5`).*

> Lời thuyết trình mẫu:
> *"A03:2025 là hạng mục mới về rủi ro Chuỗi cung ứng phần mềm. Ở phía trình duyệt, EduFlaw gắn chuẩn W3C Subresource Integrity kèm crossorigin='anonymous' cho thư viện tải từ CDN. Như Thầy/Cô thấy khi em bấm giả lập sai đúng 1 ký tự cuối của chuỗi SHA-256, trình duyệt lập tức kích hoạt lỗi đỏ trên Console và tiêu diệt file script trước khi nó kịp chạy — chặn kịch bản CDN bị tin tặc cấy mã đánh cắp phiên đăng nhập. Ở phía backend, 10/10 thư viện trong requirements.txt đều được khóa cứng phiên bản bằng toán tử ==."*

5. KỊCH BẢN DEMO TỪNG PHẦN & LỜI THUYẾT TRÌNH (BÀI 3, BÀI 4 & BÀI 5)

PHẦN 3 · BÀI 3 — OWASP A07:2025 — Chống Brute-Force & Khóa Kép 60 Giây (HTTP 429)

⏱ ~1.5 phút

1. Chuyển sang tab **Bài 3 · OWASP A07**. Giữ mặc định tài khoản mục tiêu `b24dcat001@eduflaw.com` và số lần thử `7` -> bấm **Dội Thử Hàng Loạt (Reset & Chạy)**.
2. Chỉ bảng nhật ký bên dưới: các lần đầu trả `HTTP 200` báo sai mật khẩu; từ ngưỡng **5 lần sai liên tiếp** hệ thống chuyển sang trả `HTTP 429 Too Many Requests`.
3. Chỉ bảng `3.2` bên phải (`login_attempts` trong MySQL): thấy rõ hai khóa đang bị khóa đếm ngược 60s là `ip:127.0.0.1` và `user:b24dcat001@eduflaw.com`.
4. *Kiểm chứng thêm (tùy chọn):* Bấm **🔑 Thử 1 Lần** với mật khẩu đúng `EduFlaw@2026` (hoặc bấm **Mở /login để thử đăng nhập tay ↗**) -> vẫn bị từ chối trong thời gian khóa 60s. **Xong bài nhớ bấm** **🔓 Mở Khóa Nhanh (Xóa DB)** (hoặc **🔓 Mở khóa A07** trên Thanh nổi) để các phần demo sau không bị vướng khóa.

> Lời thuyết trình mẫu:
> *"Ở bài kiểm thử A07 về Xác thực, công cụ dội 7 mật khẩu sai thật vào trang /login. Đúng tới ngưỡng 5 lần sai, hệ thống trả về HTTP 429 và lưu trạng thái khóa kép vào bảng login_attempts trong MySQL suốt 60 giây cho cả địa chỉ IP lẫn tên tài khoản. Nhờ khóa kép này, kể cả kẻ tấn công đổi IP qua proxy hoặc VPN thì tài khoản sinh viên vẫn được bảo vệ, đồng thời chỉ khóa tạm 60 giây theo khuyến nghị NIST để tránh bị lợi dụng khóa vĩnh viễn hàng loạt tài khoản gây từ chối dịch vụ."*

PHẦN 4 · BÀI 4 — OWASP A10:2025 — Xử Lý Ngoại Lệ Fail-Closed & Fuzzing 23 Payload

⏱ ~1 phút

1. Chuyển sang tab **Bài 4 · OWASP A10**. Để sẵn API mục tiêu `POST /auth/api/send-otp` (có thể thử đổi sang `POST /admin/grades/api/auto-save`).
2. Bấm nhanh một nút preset bất kỳ (như `sql_or` hay `qua_dai_10k`) rồi bấm **Gửi 1 Payload Này** để xem phản hồi đơn lẻ; sau đó bấm **💣 Bơm Toàn Bộ 23 Payload** (gồm SQLi, XSS, SSTI, Path Traversal, Null Byte, CRLF và chuỗi 10.000 ký tự `A`).
3. Chỉ khung tổng kết và cột `HTTP`, `Stack Trace?` trong bảng: đạt **0 lỗi 5xx** và **0 rò rỉ Stack Trace**, tất cả đều trả về mã lỗi client an toàn với thông báo chuẩn hóa.

> Lời thuyết trình mẫu:
> *"A10:2025 là hạng mục mới nhấn mạnh việc xử lý các tình huống ngoại lệ. Em vừa bơm đồng loạt 23 payload dị dạng từ SQL Injection, Path Traversal đến chuỗi 10.000 ký tự để ép ứng dụng phát sinh lỗi. Kết quả toàn hệ thống ghi nhận 0 lỗi 5xx và 0 lộ Stack Trace: mọi ngoại lệ đều được bắt gọn theo nguyên tắc Fail-Closed, đóng từ chối hợp lệ và giữ kín cấu trúc code lẫn truy vấn MySQL bên dưới."*

PHẦN 5 · BÀI 5 — Attack Surface Mapping — Quét 69 Endpoints Kiểm Chứng Bề Mặt Tấn Công

⏱ ~1 phút

1. Chuyển sang tab **Bài 5 · Attack Surface**. Giữ `Góc nhìn: Chưa đăng nhập` (hoặc đổi sang `Sinh viên`) -> bấm **Quét 69 Đường Dẫn**.
2. Bấm lần lượt các nút lọc phía trên bảng:
   - **302 Bảo vệ**: Các trang nghiệp vụ nội bộ đều bị chặn chuyển hướng về `/login` (hoặc dashboard khi là sinh viên).
   - **404 Không có**: Các file nhạy cảm hay bị rà quét (`/.env`, `/.git/HEAD`, `/db.sql`, `/phpmyadmin`) đều trả về 404.
   - **⚠ Cảnh báo /console**: Công cụ tự phát hiện đường dẫn debug Werkzeug đang mở trong môi trường dev để nhắc tắt khi triển khai thật.

> Lời thuyết trình mẫu:
> *"Bài 5 tự động rà quét 69 đường dẫn để lập bản đồ bề mặt tấn công dưới góc nhìn người ngoài chưa đăng nhập. Toàn bộ các endpoint quản trị và nghiệp vụ đều chuyển hướng bảo vệ HTTP 302; các tệp nhạy cảm như .env hay dump SQL trả về 404 hoàn toàn. Đặc biệt công cụ còn cảnh báo riêng route /console của chế độ dev để đội ngũ vận hành không quên tắt trước khi đưa lên server thật."*

6. PHẦN 6 (BÀI 6 · DB FORENSIC, THAO TÁC SỬA ĐIỂM THẬT) & TỔNG KẾT

PHẦN 6 · BÀI 6 **Pháp Chứng CSDL MySQL (** `AES-256-GCM` **&** `grade_audit_logs` **)** ⏱ ~2 phút

1. Chuyển sang tab **Bài 6 · DB Forensic** -> bấm 🔄 **Truy Vấn MySQL Thật**. Chỉ bảng `1) Mã hóa số CCCD (student_profiles.id_card)` : dữ liệu lưu thật trong DB toàn bộ có tiền tố `enc:v1:...` (nhãn `AES-256-GCM`). Sau đó tích vào ô `Hiện bản giải mã che mờ (đối chiếu 2 chiều)` để cho xem ứng dụng giải mã được khi có khóa trong `.env` .
2. **Thao tác sửa điểm thật trên Web App để sinh Audit Log:** Bấm nút 🖍️ **Vào /admin/grades sửa điểm (GV1) ↗** -> hệ thống chuyển thẳng sang bảng điểm lớp của `gv.001` . Sửa 1 ô điểm bất kỳ -> bấm 💾 **Lưu Toàn Bộ Bảng Điểm** -> nhập lý do (VD: "*Demo kiểm toán phúc khảo*") -> bấm 💾 **Xác nhận Lưu** .
3. Trên **Thanh nổi** góc dưới phải, bấm 📊 **Bảng Pentest Đầy Đủ ↗** quay lại tab **Bài 6** và bấm 🔄 **Truy Vấn MySQL Thật** -> bảng `2) Nhật ký sửa điểm (grade_audit_logs)` lập tức hiện bản ghi mới với người sửa (`gv.001@eduflaw.com`), thời gian, điểm cũ → điểm mới và lý do vừa nhập.
4. **Chốt toàn bộ phần demo (tùy chọn):** Bấm ⚡ **Chạy Tất Cả 6 Bài** trên header để hệ thống tự quét một lượt 6/6 bài và hiển thị tổng tiến độ `6/6 bài` .

> Lời thuyết trình mẫu (kết thúc demo):
> *"Phần cuối cùng chứng minh khả năng bảo vệ dữ liệu lưu trữ và truy vết trách nhiệm trong MySQL thật. Toàn bộ số CCCD sinh viên đều được mã hóa bằng AES-256-GCM kèm Nonce ngẫu nhiên, còn khóa SECRET_KEY nằm tách biệt trong file .env trên server ứng dụng nên kẻ gian dù có dump được database cũng không thể đọc bản rõ. Khi em vừa đóng vai Giảng viên 1 sửa một ô điểm thật trên Web và nhập lý do, bảng grade_audit_logs lập tức ghi vết đầy đủ người sửa, giờ sửa, điểm cũ — điểm mới và lý do mà giảng viên không có quyền xóa. Phần demo hệ thống EduFlaw của nhóm em xin kết thúc tại đây, kính mong nhận được câu hỏi và góp ý từ Thầy/Cô."*

7. XỬ LÝ SỰ CỐ NHANH KHI ĐANG DEMO & 5 CÂU HỎI PHẢN BIỆN BỎ TÚI

⚠️ 4 Tình Huống Hay Gặp & Cách Xử Lý Ngay (3 Giây)

- **Bị báo lỗi khóa 429 khi vừa test A07 sang bài khác:** Bấm ngay 🔐 **Mở Khóa Nhanh (Xóa DB)** ở tab A07 hoặc 🔐 **Mở khóa A07** trên Thanh nổi.
- **Vào** `/pentest` **bị 404 hoặc không thấy Thanh nổi:** Kiểm tra file `src/.env` không đặt `APP_ENV=production` ; tắt cửa sổ Flask cũ rồi chạy lại `4_CHAY_KIEM_THU_PENTEST.bat` .
- **Mất mạng Internet ở phòng hội đồng khi demo Bài 2 (SRI):** Nút 🔎 **Kiểm Toán SRI & Requirements** vẫn chạy bình thường ở chế độ đối soát cục bộ và kiểm `src/requirements.txt` (báo rõ `offline` ở dòng Hash CDN).
- **Điểm/Audit Log bị chỉnh nhiều lần cần làm sạch lại:** Đóng server và nhấp đúp `3_RESET_CSDL_DEMO.bat` để khôi phục bộ dữ liệu demo gốc.

💡 Ghi Nhớ Chuyển Chế Độ CLI Cũ & Phím Tắt Nhanh

- **Nếu thầy/cô muốn xem log Terminal kiểu cổ điển:** Tại cửa sổ đen của `4_CHAY_KIEM_THU_PENTEST.bat` , gõ phím `C` rồi `Enter` (hoặc chạy `pentest\live_demo.py --cli`).
- **Chạy thẳng 1 bài ngoài Terminal:** `src\.venv\Scripts\python.exe pentest\live_demo.py 1` (từ `1` đến `7`).
- **Thu gọn / tắt Thanh nổi trên Web thật:** Bấm dấu `_` để thu gọn thành nút tròn `Pentest Live` khi cần chụp ảnh màn hình sạch, bấm `X` để tắt hẳn thanh nổi.
- **Xem nhật ký chi tiết toàn phiên:** Kéo xuống khung **Nhật ký Thực thi Trực tiếp** và bấm 📋 **Sao chép Log** nếu cần đính kèm báo cáo.

1. SV tự sửa Cookie để đổi `user_id` **thì có xem trộm được không?** → Không. Flask Session ký HMAC-SHA256 bằng `SECRET_KEY` trên máy chủ; sửa 1 bit làm sai chữ ký số, server hủy phiên ngay.

2. Sao SV vào `/master` **bị đẩy 302 thay vì trả HTTP 403?** → Tránh xác nhận route quản trị tồn tại cho kẻ dò quét (theo khuyến nghị OWASP) và đưa người dùng gõ nhầm về đúng trang của mình.

3. Vì sao A07 chỉ khóa 60s? Nếu hacker đổi nhiều IP thì sao? → Theo NIST SP 800-63B, khóa vĩnh viễn dễ bị lợi dụng thành DoS; khóa kép theo cả `ip:` và `user:` nên đổi IP vẫn bị khóa đúng tài khoản trong 60s.

4. Tại sao thẻ SRI phải có `crossorigin="anonymous"` **?** → Chính sách Same-Origin yêu cầu thuộc tính này thì trình duyệt mới được đọc byte của file từ CDN để băm SHA-256 kiểm chứng.

5. Nếu hacker dump được MySQL thì có giải mã được số CCCD không? → Không. Trường CCCD lưu dạng `enc:v1:...` mã hóa AES-256-GCM, còn `SECRET_KEY` nằm riêng trong `.env` trên máy chủ ứng dụng.